

Code Explanation

Code Explanation: AWS Glue PySpark Data Pipeline

Overview

This is a production AWS Glue PySpark job implementing a complete data pipeline with ingestion, cleaning, Slowly Changing Dimension Type 2 (SCD2), and aggregations. It processes customer and order data from S3, applies data quality rules, tracks historical changes, and generates analytics outputs.---

Step-by-Step Breakdown

1. ****Imports and Logging Setup****

- Imports AWS SDK (boto3), PySpark, and AWS Glue libraries- Configures structured logging for pipeline observability- Sets up type hints for better code maintainability

2. ****Class Initialization (`\_\_init\_\_`)****

- Creates `SDLCWizardPipeline` class to encapsulate all pipeline logic- Initializes Spark/Glue context placeholders- Defines configuration dictionary with: - ****Source paths****: S3 locations for raw customer and order data - ****Output paths****: S3 destinations for cleaned, cataloged, and analytics data - ****SCD2 settings****: Column names for tracking record validity (IsActive, StartDate, EndDate, OpTs) - ****Hudi settings****: Configuration for Apache Hudi table format (record keys, precombine fields)

3. ****Context Initialization (`initialize\_spark\_contexts`)****

- Creates SparkContext, GlueContext, and SparkSession- Initializes AWS Glue Job with job name from command-line arguments- Configures Spark for adaptive query execution and partition tuning- Creates S3 client for bucket access validation- Handles graceful fallback if running outside Glue environment (e.g., local testing)

4. ****S3 Access Validation (`validate\_s3\_access`)****

- Parses S3 URI to extract bucket and prefix- Uses boto3 `head\_bucket` to verify read/write permissions- Returns boolean success/failure to prevent runtime errors on inaccessible paths

5. ****Safe Data Reading (`read\_data\_safe`)****

- Validates S3 access before attempting read- Supports multiple formats (CSV, Parquet, JSON)- Applies schema enforcement if provided- Sets CSV defaults: header parsing, permissive mode for malformed records- Normalizes all column names to lowercase for consistency- Logs record counts and returns None on failure (defensive programming)

6. ****Safe Data Writing (`write\_data\_safe`)****

- Validates S3 access before write- Supports standard formats (Parquet, CSV) and Hudi for SCD2- Applies Hudi-specific options when format is 'hudi'- Supports partitioning for performance optimization- Returns boolean success indicator

7. **Schema Definitions (`get\_customer\_schema`, `get\_order\_schema`)**

- Defines explicit schemas matching source data structure- Customer schema: ID, name, contact info, address, registration date, status- Order schema: ID, customer reference, date, amount, status, product, quantity, payment method- Uses appropriate data types (StringType, DecimalType, TimestampType, IntegerType)

8. **Data Cleaning (`clean\_data`)**

Implements FR-CLEAN-001 and TR-CLEAN-001:
Step 1: Removes rows with NULL values in critical fields (e.g., customer_id, order_id)- **Step 2:** Filters out string representations of NULL ('Null', 'NULL', 'null', 'None', '')- **Step 3:** Deduplicates based on primary key (first field in critical_fields list)- Logs counts at each stage for data quality auditing

9. **SCD Type 2 Column Addition (`add\_scd2\_columns`)**

Implements FR-SCD2-001 and TR-SCD2-001:
Adds four tracking columns: - **IsActive:** Boolean flag (True for current records) - **StartDate:** When this version became active (current timestamp) - **EndDate:** When this version expires (9999-12-31 for active records) - **OpTs:** Operation timestamp for change tracking- Handles both new records and updates to existing records

10. **SCD Type 2 Upsert Logic (`perform\_scd2\_upsert`)**

Core SCD2 implementation:
If no existing data: Treats all records as new inserts- **If existing data present:** 1. Filters to active records only (IsActive = True) 2. Left joins new data with existing on primary key 3. Identifies three categories: - **New records:** customer_id not in existing data - **Changed records:** customer_id exists but compare_columns differ - **Unchanged records:** customer_id exists with identical data 4. For changed records: - Expires old version (sets IsActive=False, EndDate=current timestamp) - Creates new version (sets IsActive=True, StartDate=current timestamp) 5. Combines all records into single DataFrame- Preserves full history while maintaining current state

11. **Customer Aggregate Spend (`calculate\_customer\_aggregate\_spend`)**

Implements FR-AGG-001 and TR-AGG-001:
Groups orders by customer_id- Calculates: - **total_spend:** Sum of all order amounts - **order_count:** Number of orders placed - **avg_spend_per_order:** Average order value- Joins with customer data to include customer_name- Adds calculation_date timestamp for audit trail

12. **Order Summary Generation (`generate\_order\_summary`)**

Implements FR-SUMMARY-001:
Aggregates order data per customer: - **total_orders:** Count of orders - **total_amount:** Sum of order amounts - **avg_order_amount:** Average order value - **first_order_date:** Earliest order timestamp - **last_order_date:** Most recent order timestamp- Adds summary_date for tracking when summary was generated

13. **Pipeline Execution (`run\_pipeline`)**

Orchestrates entire workflow:
1. **Initialize:** Set up Spark/Glue contexts
2. **Ingest:** Read customer and order CSV files from S3
3. **Clean:** Apply data quality rules to both datasets
4. **SCD2 Processing:** - Read existing customer data (if available) - Apply SCD Type 2 logic to track changes -

Write to Hudi format with Glue Catalog sync5. ****Write Orders****: Save cleaned order data as Parquet6. ****Aggregations****: - Calculate customer aggregate spend - Generate order summary statistics7. ****Commit****: Finalize Glue job if running in AWS environmentReturns True on success, False on any failure.

14. ****Main Entry Point (`main`)****

- Creates pipeline instance- Executes pipeline- Exits with code 0 (success) or 1 (failure) for job orchestration---

Key Design Patterns

1. ****Defensive Programming****: All I/O operations validate access and return None/False on failure2. ****Logging****: Comprehensive logging at each stage for debugging and monitoring3. ****Separation of Concerns****: Each method handles one specific responsibility4. ****Configuration-Driven****: All paths and settings in config dictionary for easy modification5. ****SCD Type 2****: Full historical tracking with active/inactive flags and date ranges6. ****Hudi Integration****: Uses Apache Hudi for efficient upserts and time-travel queries---

Data Flow Summary

```
Raw S3 Data → Validation → Cleaning → SCD2 Processing → Hudi/Parquet Storage ↓
Aggregations → Analytics Outputs
```